# AI & MACHINE LEARNING PRACTICAL EXAM GUIDE

33 Beginner-Friendly Programs | Theory | Line-by-Line Guidance | Viva Preparation

Prepared from the supplied 13-page AI/ML problem statement sheet
Experiments 1-9 | Questions 1-33

*Beginner-first approach: understand the idea, run the code, explain the important lines, and practise the viva answers.*

# How to use this guide

- Study one question at a time. First read the theory, then run the matching .py file from the accompanying ZIP, then explain the important lines without looking.
- For graph/search and CSP questions, trace a tiny example by hand. Be able to explain the data structure, the condition, and the stopping condition.
- For machine-learning questions, remember the common sequence: load data -> inspect -> prepare X and y -> split -> fit -> predict -> evaluate.
- CSV-based questions depend on the exact headers in your lab dataset. Each question below states the expected file and columns. Change only the filename/column names when needed.
- The code uses beginner-friendly examples. Where a short example simplifies a real-world pipeline, the notes identify the more rigorous alternative.

## Common installation

```
pip install pandas numpy matplotlib scikit-learn
pip install mlxtend       # Q29
pip install tensorflow   # Q30 and Q31
```

## Common ML terms you should know

| Feature (X) | An input column used by the model. |
|---|---|
| **Target (y)** | The output column the model should predict. |
| **Training data** | Examples used to learn model parameters. |
| **Testing data** | Held-out examples used to evaluate the trained model. |
| **Classification** | Predicting a category, such as spam/not spam or pass/fail. |
| **Regression** | Predicting a continuous numerical value, such as salary or house price. |
| **Clustering** | Grouping unlabelled data points based on similarity. |
| **Overfitting** | Learning training-specific patterns that do not generalize well. |
| **Data leakage** | Allowing test-set or future information to influence model fitting. |
| **Hyperparameter** | A setting chosen before/during training, such as K in KNN or max_depth in a tree. |

# Question index


## Experiment 1: Informed Search Algorithms

- Q1: A* Search for Route Finding
- Q2: Greedy Best-First Search
- Q3: Robot Navigation Using A* on an 8 x 8 Grid

## Experiment 2: Constraint Satisfaction Problems (CSP)

- Q4: N-Queens Using Backtracking
- Q5: Sudoku Solver Using Backtracking
- Q6: Examination Timetable Scheduling Using CSP

## Experiment 3: Game Playing and Puzzle Solving

- Q7: Tic-Tac-Toe Using Minimax
- Q8: 8-Puzzle Using A* Search

## Experiment 4: EDA, PCA and Regression

- Q9: Uber Fare Prediction Using PCA and EDA
- Q10: House Price Prediction Using Linear Regression
- Q11: Student Performance Prediction Using Regression
- Q12: Salary Estimation Using Linear Regression
- Q13: Compare Regression Models With and Without PCA

## Experiment 5: Decision Tree, KNN and Naive Bayes

- Q14: Loan Approval Prediction Using Decision Tree
- Q15: Customer Churn Prediction Using Decision Tree
- Q16: Disease Classification Using Decision Tree
- Q17: Decision Tree Optimization: Entropy vs Gini
- Q18: Iris Flower Classification Using KNN
- Q19: Student Performance Classification Using KNN
- Q20: Diabetes Classification Using KNN
- Q21: Email or SMS Spam Detection Using Naive Bayes
- Q22: Iris Classification Using Gaussian Naive Bayes vs Decision Tree

## Experiment 6: Ensemble Learning and K-Means Clustering

- Q23: Compare Random Forest and Decision Tree
- Q24: Titanic Survival Prediction Using Random Forest
- Q25: Customer Segmentation Using K-Means
- Q26: Student Grouping Using K-Means
- Q27: Mall Customer Clustering
- Q28: Evaluate K-Means for Different Cluster Counts

## Experiment 7: Apriori Association Rule Mining

- Q29: Market Basket Analysis Using Apriori

## Experiment 8: Artificial Neural Networks

- Q30: Bank Customer Churn Prediction Using ANN
- Q31: Student Pass/Fail Prediction Using ANN

## Experiment 9: Integrated Classification and ML Pipeline

- Q32: Compare Decision Tree, KNN and Naive Bayes
- Q33: End-to-End Machine Learning Pipeline

# Experiment 1: Informed Search Algorithms

## Q1. A* Search for Route Finding

**Problem focus:** A* Search for Route Finding.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

A* is an informed search algorithm used to find a low-cost path from a start node to a goal node. It evaluates a node using f(n) = g(n) + h(n), where g(n) is the actual cost from the start and h(n) is an estimated cost from the current node to the goal. A priority queue expands the node with the smallest f value. With an admissible heuristic (one that never overestimates the remaining cost), A* can find an optimal path when the usual graph-search conditions are satisfied.

### Beginner-level Python program (with comments)

```
import heapq
import math

# Each city is connected to its neighbours with a travel cost.
graph = {
    "Pune": {"Mumbai": 150, "Nashik": 210, "Satara": 110},
    "Mumbai": {"Pune": 150, "Surat": 280, "Nashik": 170},
    "Nashik": {"Pune": 210, "Mumbai": 170, "Aurangabad": 190},
    "Aurangabad": {"Nashik": 190, "Nagpur": 500},
    "Nagpur": {"Aurangabad": 500, "Satara": 700},
    "Satara": {"Pune": 110, "Nagpur": 700, "Kolhapur": 180},
    "Kolhapur": {"Satara": 180, "Goa": 230},
    "Goa": {"Kolhapur": 230, "Surat": 650},
    "Surat": {"Mumbai": 280, "Goa": 650, "Nagpur": 850}
}

# Approximate map coordinates; straight-line distance is our heuristic.
coordinates = {
    "Pune": (0, 0), "Mumbai": (120, 0), "Nashik": (110, 140),
    "Aurangabad": (300, 150), "Nagpur": (750, 230),
    "Satara": (-60, -100), "Kolhapur": (-150, -250),
    "Goa": (-300, -350), "Surat": (390, -20)
}

def heuristic(city, goal):
    # Euclidean distance estimates the distance remaining to the goal.
    x1, y1 = coordinates[city]
    x2, y2 = coordinates[goal]
    # The 0.5 factor keeps this straight-line estimate conservative for the sample edge costs.
    return 0.5 * math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)

def a_star(start, goal):
    # Heap items are (estimated total cost, actual cost, current city).
    queue = [(heuristic(start, goal), 0, start)]
    cost = {start: 0}          # Best known g(n) for each city.
    parent = {start: None}     # Used later to rebuild the route.
    explored = []

    while queue:
        f, current_cost, current = heapq.heappop(queue)

        # Ignore an old queue entry if a cheaper route was found later.
        if current_cost != cost[current]:
            continue
```

```
        explored.append(current)
        if current == goal:
            break

        for neighbour, edge_cost in graph[current].items():
            new_cost = current_cost + edge_cost
            if neighbour not in cost or new_cost < cost[neighbour]:
                cost[neighbour] = new_cost
                parent[neighbour] = current
                new_f = new_cost + heuristic(neighbour, goal)
                heapq.heappush(queue, (new_f, new_cost, neighbour))

    if goal not in parent:
        return None, None, explored

    # Follow parent links backwards, then reverse to get start -> goal.
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path, cost[goal], explored

# Read and validate the user\'s source and destination.
print("Available cities:", ", ".join(graph))
start = input("Enter source city: ").strip().title()
goal = input("Enter destination city: ").strip().title()

if start not in graph or goal not in graph:
    print("Invalid city name.")
elif start == goal:
    print("Source and destination are the same. Cost = 0")
else:
    path, total_cost, explored = a_star(start, goal)
    if path:
        print("Route:", " -> ".join(path))
        print("Total path cost:", total_cost)
        print("Nodes explored:", len(explored), explored)
    else:
        print("No route found.")
```

### Important lines explained

- `graph` is an adjacency list: each city maps to its neighbours and edge costs.
- `heuristic()` estimates the remaining distance; the goal is passed in, so the user can choose any destination.
- `queue` is a min-heap. A* removes the entry with the smallest `f = g + h` first.
- `cost` stores the best actual cost discovered so far. `parent` stores the previous city for path reconstruction.
- The neighbour loop relaxes an edge when it discovers a cheaper route.
- The final loop follows parent pointers backwards and reverses the list.
- `explored` records the cities removed from the priority queue; the printed count is the number expanded.

### Alternatives and exam-friendly variations

- Use `queue.PriorityQueue` instead of `heapq` for a thread-safe priority queue; `heapq` is simpler for a practical.
- Use road-distance heuristic values in a dictionary if supplied by the question. A heuristic must be calculated for the chosen goal.
- Use Dijkstra by setting every heuristic to zero: `f(n) = g(n)`.

### Likely viva questions and answers

**Q. What is A*?** A. An informed search algorithm that uses both the path cost so far and an estimate of the remaining cost.

**Q. What is f(n)?** A. `f(n) = g(n) + h(n)`.

**Q. What is an admissible heuristic?** A. A heuristic that never overestimates the true minimum remaining cost.

**Q. Why use a priority queue?** A. It selects the frontier node with the smallest estimated total cost efficiently.

## Q2. Greedy Best-First Search

**Problem focus:** Greedy Best-First Search.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Greedy Best-First Search is an informed search algorithm that selects the next node using only its heuristic value h(n), which estimates how close the node is to the goal. It uses a priority queue and a visited set to avoid repeatedly processing nodes. It can reach a goal quickly, but it does not guarantee the shortest path because it ignores the cost already travelled.

### Beginner-level Python program (with comments)

```
import heapq
import math

graph = {
    "Pune": {"Mumbai": 150, "Nashik": 210, "Satara": 110},
    "Mumbai": {"Pune": 150, "Surat": 280, "Nashik": 170},
    "Nashik": {"Pune": 210, "Mumbai": 170, "Aurangabad": 190},
    "Aurangabad": {"Nashik": 190, "Nagpur": 500},
    "Nagpur": {"Aurangabad": 500, "Satara": 700},
    "Satara": {"Pune": 110, "Nagpur": 700, "Kolhapur": 180},
    "Kolhapur": {"Satara": 180, "Goa": 230},
    "Goa": {"Kolhapur": 230, "Surat": 650},
    "Surat": {"Mumbai": 280, "Goa": 650, "Nagpur": 850}
}
coordinates = {
    "Pune": (0, 0), "Mumbai": (120, 0), "Nashik": (110, 140),
    "Aurangabad": (300, 150), "Nagpur": (750, 230),
    "Satara": (-60, -100), "Kolhapur": (-150, -250),
    "Goa": (-300, -350), "Surat": (390, -20)
}

def heuristic(city, goal):
    # Straight-line distance is used only to guide the search.
    x1, y1 = coordinates[city]
    x2, y2 = coordinates[goal]
    # The 0.5 factor keeps this straight-line estimate conservative for the sample edge costs.
    return 0.5 * math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)

def greedy_best_first(start, goal):
    # Priority is h(n), not g(n) + h(n).
    queue = [(heuristic(start, goal), start)]
    visited = set()
    parent = {start: None}
    explored = []

    while queue:
        h, current = heapq.heappop(queue)
```

```
        if current in visited:
            continue
        visited.add(current)
        explored.append(current)

        if current == goal:
            break

        for neighbour in graph[current]:
            if neighbour not in visited and neighbour not in parent:
                parent[neighbour] = current
                heapq.heappush(queue, (heuristic(neighbour, goal), neighbour))

    if goal not in visited:
        return None, None, explored

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()

    # Calculate the route's actual cost after finding the route.
    total_cost = 0
    for i in range(len(path) - 1):
        total_cost += graph[path[i]][path[i + 1]]
    return path, total_cost, explored

print("Available cities:", ", ".join(graph))
start = input("Enter source city: ").strip().title()
goal = input("Enter destination city: ").strip().title()

if start not in graph or goal not in graph:
    print("Invalid city name.")
elif start == goal:
    print("Route:", start, "| Cost: 0 | Nodes explored: 1")
else:
    path, total_cost, explored = greedy_best_first(start, goal)
    if path:
        print("Route:", " -> ".join(path))
        print("Total path cost:", total_cost)
        print("Nodes explored:", len(explored), explored)
    else:
        print("No route found.")
```

### Important lines explained

- The graph and heuristic are similar to Q1, but the selection rule is different.
- `queue` stores `(h, city)`, so only the heuristic controls which city is selected.
- `visited` prevents a city from being expanded more than once.
- `parent` records the route used to reach each discovered city.
- After finding the route, a separate loop adds the real edge costs. Greedy selection itself does not consider those costs.
- The route is not guaranteed to be shortest; this is an important oral-exam distinction from A*.

### Alternatives and exam-friendly variations

- Replace Euclidean distance with a supplied heuristic table such as `h = {"Pune": 10, ...}`.
- A list can be sorted after adding nodes, but `heapq` is more efficient for repeated minimum selection.
- A* uses `g + h`; Greedy Best-First uses only `h`.

### Likely viva questions and answers

**Q. What does Greedy Best-First Search minimize?** A. The heuristic value h(n) at each selection.

**Q. Does it guarantee the shortest path?** A. No. It ignores the cost already travelled.

**Q. Difference between A* and Greedy?** A. A* uses g(n)+h(n), while Greedy uses only h(n).

**Q. Why maintain a visited set?** A. To avoid expanding the same node repeatedly.

## Q3. Robot Navigation Using A* on an 8 x 8 Grid

**Problem focus:** Robot Navigation Using A* on an 8 x 8 Grid.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Grid-based robot navigation represents free cells and blocked cells in a two-dimensional grid. A* searches for a path from a start cell to a goal cell. The Manhattan distance, |row1-row2| + |col1-col2|, is suitable when movement is restricted to up, down, left and right. The path length is the number of moves between consecutive cells.

### Beginner-level Python program (with comments)

```
import heapq

# 0 means free cell; 1 means obstacle. The grid has 8 rows and 8 columns.
grid = [
    [0, 0, 0, 0, 1, 0, 0, 0],
    [1, 1, 0, 0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 0],
    [0, 1, 0, 0, 0, 1, 0, 1],
    [0, 0, 0, 1, 0, 0, 0, 0],
    [1, 1, 0, 1, 0, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (7, 7)

def manhattan(a, b):
    # Number of horizontal plus vertical steps in an obstacle-free grid.
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar_grid(start, goal):
    # Heap entries: (f score, g score, cell).
    queue = [(manhattan(start, goal), 0, start)]
    cost = {start: 0}
    parent = {start: None}
    explored = []
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    while queue:
        f, g, current = heapq.heappop(queue)
        if g != cost[current]:
            continue
        explored.append(current)
        if current == goal:
            break

        for dr, dc in directions:
            nr = current[0] + dr
            nc = current[1] + dc
            neighbour = (nr, nc)
```

```
        # Check grid boundary and obstacle before using the cell.
        if 0 <= nr < 8 and 0 <= nc < 8 and grid[nr][nc] == 0:
            new_cost = g + 1
            if neighbour not in cost or new_cost < cost[neighbour]:
                cost[neighbour] = new_cost
                parent[neighbour] = current
                priority = new_cost + manhattan(neighbour, goal)
                heapq.heappush(queue, (priority, new_cost, neighbour))

    if goal not in parent:
        return None, explored

    path = []
    cell = goal
    while cell is not None:
        path.append(cell)
        cell = parent[cell]
    path.reverse()
    return path, explored

path, explored = astar_grid(start, goal)
if path is None:
    print("No valid path exists.")
else:
    path_cells = set(path)
    print("Grid: S=start, G=goal, #=obstacle, *=path, .=free")
    for r in range(8):
        row = []
        for c in range(8):
            cell = (r, c)
            if cell == start:
                row.append("S")
            elif cell == goal:
                row.append("G")
            elif grid[r][c] == 1:
                row.append("#")
            elif cell in path_cells:
                row.append("*")
            else:
                row.append(".")
        print(" ".join(row))
    print("Path:", path)
    print("Path length:", len(path) - 1, "moves")
    print("Cells explored:", len(explored))
```

### Important lines explained

- `grid[r][c] == 1` means the robot cannot enter that cell.
- `directions` defines four-way movement. Diagonal movement is deliberately not allowed.
- `manhattan()` is the heuristic; it is appropriate for four-direction movement with equal move costs.
- `new_cost = g + 1` means every grid move costs one.
- A neighbour is pushed only when it is free and the new route is cheaper.
- The printed path length is `len(path) - 1`, because a path with N cells contains N-1 moves.

### Alternatives and exam-friendly variations

- For diagonal movement, use an appropriate diagonal heuristic such as octile distance and add diagonal directions.
- Use Euclidean distance if diagonal movement is allowed with suitable movement costs.
- A BFS queue is enough for an unweighted grid if no heuristic is required.

### Likely viva questions and answers

**Q. What is Manhattan distance?** A. The sum of absolute row and column differences.

**Q. Why use A* for a grid?** A. It combines the distance travelled and an estimate of the distance remaining.

**Q. What is an obstacle?** A. A blocked cell that cannot be included in the route.

**Q. How is path length calculated?** A. Number of path cells minus one, when each move costs one.

# Experiment 2: Constraint Satisfaction Problems (CSP)

## Q4. N-Queens Using Backtracking

**Problem focus:** N-Queens Using Backtracking.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

The N-Queens problem is a Constraint Satisfaction Problem (CSP). Each variable represents the row position of a queen in one column. The domain of each variable is the set of rows 0 to N-1. Constraints require that no two queens share a row or diagonal; because exactly one queen is placed in each column, column conflicts are avoided by construction. Backtracking places one queen at a time and reverses a choice when it causes a conflict.

### Beginner-level Python program (with comments)

```
def is_safe(positions, row, col):
    # Check every queen already placed in an earlier column.
    for old_col in range(col):
        old_row = positions[old_col]
        if old_row == row:
            return False  # Same row is not allowed.
        if abs(old_row - row) == abs(old_col - col):
            return False  # Equal row/column difference means same diagonal.
    return True

def solve_n_queens(n, positions, col):
    # All columns have a queen: a complete solution has been found.
    if col == n:
        return True

    # Try each row as a possible position in this column.
    for row in range(n):
        if is_safe(positions, row, col):
            positions[col] = row  # Make a tentative choice.
            if solve_n_queens(n, positions, col + 1):
                return True
            positions[col] = -1  # Undo the choice (backtrack).
    return False

n = int(input("Enter N (minimum 4): "))
if n < 4:
    print("Please enter N >= 4.")
else:
    positions = [-1] * n  # positions[col] stores the queen's row.
    if solve_n_queens(n, positions, 0):
        print("One valid board configuration:")
        for row in range(n):
            line = []
            for col in range(n):
                line.append("Q" if positions[col] == row else ".")
            print(" ".join(line))
        print("Solution found.")
    else:
        print("No solution found.")
```

### Important lines explained

- `positions[col]` stores the row occupied by the queen in that column.
- `is_safe()` checks row and diagonal conflicts against already placed queens.

- `col == n` is the base case: every column has a queen.
- The recursive call moves to the next column.
- Setting the position back to -1 removes a failed tentative choice; this is backtracking.
- One queen per column is guaranteed by placing exactly one queen before moving to the next column.

### Alternatives and exam-friendly variations

- Use sets for occupied rows, `row-col` diagonals and `row+col` diagonals for faster conflict checks.
- Use the Minimum Remaining Values (MRV) heuristic to choose the most constrained variable first in a general CSP.
- For all solutions, do not return immediately after the first successful recursive call; collect each solution instead.

### Likely viva questions and answers

**Q. What are the CSP components?** A. Variables, domains and constraints.

**Q. What is backtracking?** A. A depth-first method that tries a choice and reverses it when it leads to failure.

**Q. How do we detect a diagonal conflict?** A. Two queens share a diagonal when the absolute row difference equals the absolute column difference.

**Q. Why is one queen per column automatic?** A. The algorithm places one queen in a column and then advances to the next column.

## Q5. Sudoku Solver Using Backtracking

**Problem focus:** Sudoku Solver Using Backtracking.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Sudoku is a CSP in which each empty cell is a variable and its domain is the digits 1 to 9. The constraints require each digit to appear only once in every row, column and 3 x 3 subgrid. Backtracking selects an empty cell, tries legal digits and recursively continues. If no digit works, it resets the cell and tries another choice. The program must also detect an unsolvable puzzle.

### Beginner-level Python program (with comments)

```
# 0 represents an empty Sudoku cell. Keep True to run the sample puzzle.
USE_SAMPLE = True
if USE_SAMPLE:
    board = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
    ]
else:
    # Enter 9 lines; each line must contain 9 integers. Use 0 for blanks.
    board = []
    for row_number in range(9):
        row = list(map(int, input(f"Enter row {row_number + 1}: ").split()))
        if len(row) != 9 or any(value < 0 or value > 9 for value in row):
            raise ValueError("Each row must contain 9 values from 0 to 9.")
        board.append(row)
```

```
def find_empty():
    # Return the first empty cell as (row, column); return None if full.
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:
                return row, col
    return None

def is_valid(row, col, number):
    # Check the selected row.
    if number in board[row]:
        return False

    # Check the selected column.
    for r in range(9):
        if board[r][col] == number:
            return False

    # Find the top-left corner of the cell's 3 x 3 subgrid.
    box_row = (row // 3) * 3
    box_col = (col // 3) * 3
    for r in range(box_row, box_row + 3):
        for c in range(box_col, box_col + 3):
            if board[r][c] == number:
                return False
    return True

def solve():
    empty = find_empty()
    if empty is None:
        return True  # No empty cells remain: puzzle solved.

    row, col = empty
    for number in range(1, 10):
        if is_valid(row, col, number):
            board[row][col] = number  # Try a valid number.
            if solve():
                return True
            board[row][col] = 0  # Undo a failed attempt.
    return False

def validate_solution():
    # Every row, column and 3 x 3 box must contain 1 through 9.
    expected = set(range(1, 10))
    for i in range(9):
        if set(board[i]) != expected:
            return False
        if {board[r][i] for r in range(9)} != expected:
            return False
    for br in range(0, 9, 3):
        for bc in range(0, 9, 3):
            values = {board[r][c] for r in range(br, br + 3)
                      for c in range(bc, bc + 3)}
            if values != expected:
                return False
    return True

if solve() and validate_solution():
    print("Solved Sudoku:")
    for row in board:
        print(*row)
```

```
else:
    print("The puzzle has no valid solution.")
```

### Important lines explained

- The grid uses 0 to represent an empty cell. Set `USE_SAMPLE = False` to enter nine rows from the keyboard.
- `find_empty()` selects the next variable that has not been assigned.
- `is_valid()` checks the three Sudoku constraints before assigning a digit.
- `// 3 * 3` finds the first row or column of the relevant 3 x 3 box.
- `solve()` tries digits 1 through 9 and recursively solves the remaining cells.
- If a later cell cannot be solved, the current cell is reset to 0.
- `validate_solution()` independently checks the completed rows, columns and boxes.

### Alternatives and exam-friendly variations

- Use the Minimum Remaining Values method: select the empty cell with the fewest legal candidates, rather than the first empty cell.
- Use bit masks or sets to make row, column and box checks faster.
- A Sudoku-solving library can be used in applications, but backtracking is easier to explain in a practical oral.

### Likely viva questions and answers

**Q. What is the domain of an empty Sudoku cell?** A. Digits 1 to 9, subject to the constraints.

**Q. What are the three constraints?** A. No repeated digit in a row, column or 3 x 3 subgrid.

**Q. What represents an empty cell in this code?** A. Zero.

**Q. How does the program detect failure?** A. When no candidate digit works for an empty cell, the recursion backtracks; if all choices fail, it returns False.

## Q6. Examination Timetable Scheduling Using CSP

**Problem focus:** Examination Timetable Scheduling Using CSP.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Examination scheduling is a CSP. Each examination is a variable, and its domain is the available time slots. Two examinations conflict when at least one student is enrolled in both; conflicting examinations cannot use the same slot. Backtracking assigns slots one examination at a time and rejects assignments that violate a conflict constraint. If no assignment is possible, the available slots are insufficient for this set of constraints.

### Beginner-level Python program (with comments)

```
# Six exams and the students enrolled in each exam (sample data).
enrollments = {
    "Maths": {"S1", "S2"},
    "DBMS": {"S2", "S3"},
    "Python": {"S4", "S5"},
    "Networks": {"S3", "S6"},
    "OS": {"S5", "S7"},
    "AI": {"S1", "S6"}
}
slots = ["Slot 1", "Slot 2", "Slot 3", "Slot 4"]

# Build a conflict set for every exam.
conflicts = {exam: set() for exam in enrollments}
exams = list(enrollments)
for i in range(len(exams)):
```

```
    for j in range(i + 1, len(exams)):
        first = exams[i]
        second = exams[j]
        if enrollments[first] & enrollments[second]:
            conflicts[first].add(second)
            conflicts[second].add(first)

def schedule(index, timetable):
    # All exams have been assigned a slot.
    if index == len(exams):
        return True

    exam = exams[index]
    for slot in slots:
        # A conflicting exam must not already use this slot.
        clash = False
        for other_exam in conflicts[exam]:
            if timetable.get(other_exam) == slot:
                clash = True
                break

        if not clash:
            timetable[exam] = slot  # Assign this exam temporarily.
            if schedule(index + 1, timetable):
                return True
            del timetable[exam]  # Backtrack if later exams cannot fit.
    return False

print("Exam conflict pairs:")
for exam in exams:
    for other in conflicts[exam]:
        if exams.index(exam) < exams.index(other):
            print(exam, "-", other)

result = {}
if schedule(0, result):
    print("\nValid timetable:")
    for exam in exams:
        print(exam, "->", result[exam])
    print("No conflicting exams share a slot.")
else:
    print("No valid timetable: available slots are insufficient.")
```

### Important lines explained

- `enrollments` maps each exam to the set of students taking it.
- The set intersection `&` finds students who are enrolled in both exams.
- `conflicts` stores the exams that cannot run simultaneously.
- `schedule()` tries each time slot for the current exam.
- Before assigning a slot, it checks whether any conflicting exam already uses that slot.
- `del timetable[exam]` removes an unsuccessful assignment so another slot can be tried.

### Alternatives and exam-friendly variations

- Sort exams by descending number of conflicts (degree heuristic) before scheduling to expose difficult exams earlier.
- Use a dictionary of student-to-exams and build conflicts from it.
- Use graph colouring terminology: exams are vertices, conflicts are edges, and time slots are colours.

### Likely viva questions and answers

**Q. How is a conflict identified?** A. Two exams conflict if their enrollment sets have a non-empty intersection.

**Q. What does a time slot represent in graph colouring?** A. A colour assigned to a vertex.

**Q. Why use backtracking?** A. A slot assignment may block later exams, so the algorithm must undo it and try another slot.

**Q. What happens when no solution is found?** A. The current number of slots cannot satisfy all the conflict constraints for the given data.

# Experiment 3: Game Playing and Puzzle Solving

## Q7. Tic-Tac-Toe Using Minimax

**Problem focus:** Tic-Tac-Toe Using Minimax.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Minimax is a decision-making algorithm for two-player, turn-based, zero-sum games. It assumes both players choose the best possible move. In this program, the human is X (the minimizing player) and the computer is O (the maximizing player). A terminal board receives a score: +1 for an O win, -1 for an X win and 0 for a draw. The algorithm explores possible moves and selects the move with the best guaranteed score.

### Beginner-level Python program (with comments)

```
# Empty cells contain a space; the human plays X and the computer plays O.
board = [" "] * 9

def show_board():
    print()
    for row in range(3):
        print(" | ".join(board[row * 3:(row + 1) * 3]))
        if row < 2:
            print("---------")

def winner(player):
    # All possible rows, columns and diagonals.
    wins = [(0, 1, 2), (3, 4, 5), (6, 7, 8),
            (0, 3, 6), (1, 4, 7), (2, 5, 8),
            (0, 4, 8), (2, 4, 6)]
    return any(board[a] == board[b] == board[c] == player for a, b, c in wins)

def minimax(is_maximizing):
    # Terminal states receive a score.
    if winner("O"):
        return 1
    if winner("X"):
        return -1
    if " " not in board:
        return 0

    if is_maximizing:  # Computer tries to maximize the score.
        best = -float("inf")
        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                best = max(best, minimax(False))
                board[i] = " "  # Undo the test move.
        return best
    else:  # Human is assumed to minimize the computer's score.
        best = float("inf")
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                best = min(best, minimax(True))
                board[i] = " "
        return best

def computer_move():
    best_score = -float("inf")
```

```
    chosen = None
    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(False)
            board[i] = " "
            if score > best_score:
                best_score = score
                chosen = i
    if chosen is not None:
        board[chosen] = "O"

print("Positions are numbered 1 to 9 from left to right, top to bottom.")
while True:
    show_board()
    move = int(input("Your move (1-9): ")) - 1
    if move < 0 or move > 8 or board[move] != " ":
        print("Invalid or occupied position. Try again.")
        continue
    board[move] = "X"

    if winner("X"):
        show_board()
        print("You win.")
        break
    if " " not in board:
        show_board()
        print("Draw.")
        break

    computer_move()
    if winner("O"):
        show_board()
        print("Computer wins.")
        break
    if " " not in board:
        show_board()
        print("Draw.")
        break
```

### Important lines explained

- The board is a list of nine cells; positions 0 through 8 correspond to the 3 x 3 board.
- `winner()` checks the eight possible winning lines.
- `minimax(True)` means the computer O is choosing; `minimax(False)` means X is choosing.
- At a terminal state, the function returns +1, -1 or 0.
- Each recursive branch makes a temporary move, evaluates it, and resets the cell to a space.
- `computer_move()` tests every legal O move and keeps the move with the highest score.

### Alternatives and exam-friendly variations

- Add alpha-beta pruning to avoid exploring branches that cannot change the final decision.
- Use depth-aware scores (for example, `10-depth`) to prefer faster wins and slower losses.
- For larger games, use a depth limit and an evaluation function; tic-tac-toe is small enough to search completely.

### Likely viva questions and answers

**Q. What is Minimax?** A. A game-tree search method that chooses the best move assuming both players play optimally.

**Q. Why are there maximizing and minimizing players?** A. One player tries to maximize the score while the opponent tries to minimize it.

**Q. What do +1, -1 and 0 represent here?** A. Computer win, human win and draw.

**Q. What is alpha-beta pruning?** A. An optimization that skips branches that cannot affect the Minimax decision.

## Q8. 8-Puzzle Using A* Search

**Problem focus:** 8-Puzzle Using A* Search.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

The 8-puzzle contains eight numbered tiles and one blank cell on a 3 x 3 board. A state is represented by a tuple of nine values, where 0 is the blank. A* uses f(n) = g(n) + h(n). The Manhattan heuristic adds the row and column distance of every tile from its goal position, excluding the blank. A visited/best-cost dictionary prevents unnecessary repeated-state processing.

### Beginner-level Python program (with comments)

```
import heapq

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def manhattan(state):
    # Sum each numbered tile's distance from its goal position.
    distance = 0
    for index, tile in enumerate(state):
        if tile == 0:
            continue  # The blank is not counted.
        goal_index = goal.index(tile)
        row1, col1 = divmod(index, 3)
        row2, col2 = divmod(goal_index, 3)
        distance += abs(row1 - row2) + abs(col1 - col2)
    return distance

def neighbours(state):
    # Generate states by moving the blank one legal step.
    zero = state.index(0)
    row, col = divmod(zero, 3)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    result = []
    for dr, dc in moves:
        nr, nc = row + dr, col + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_index = nr * 3 + nc
            new_state = list(state)
            new_state[zero], new_state[new_index] = new_state[new_index], new_state[zero]
            result.append(tuple(new_state))
    return result

def solve_a_star():
    queue = [(manhattan(start), 0, start)]  # (f, g, state)
    cost = {start: 0}
    parent = {start: None}
```

```
    explored = 0

    while queue:
        f, g, state = heapq.heappop(queue)
        if g != cost[state]:
            continue
        explored += 1
        if state == goal:
            path = []
            current = goal
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1], explored

        for next_state in neighbours(state):
            new_cost = g + 1
            if next_state not in cost or new_cost < cost[next_state]:
                cost[next_state] = new_cost
                parent[next_state] = state
                heapq.heappush(queue, (new_cost + manhattan(next_state),
                                       new_cost, next_state))
    return None, explored

def display(state):
    for i in range(0, 9, 3):
        print(state[i:i + 3])
    print()

path, explored = solve_a_star()
if path:
    for step, state in enumerate(path):
        print("Step", step)
        display(state)
    print("Number of moves:", len(path) - 1)
    print("States explored:", explored)
else:
    print("No solution found for this starting state.")
```

### Important lines explained

- A state is a tuple, making it hashable and suitable as a dictionary key.
- `manhattan()` locates each tile's goal index and adds its row and column distance.
- `neighbours()` swaps the blank with each legal adjacent tile.
- `cost` stores the best number of moves from the start to each state.
- `parent` stores the previous state so the final sequence can be reconstructed.
- The algorithm stops when the goal tuple is removed from the priority queue.

### Alternatives and exam-friendly variations

- Add an inversion-parity check before searching to reject unsolvable 3 x 3 puzzle states early.
- Use a set for closed states; a best-cost dictionary is more informative because it also tracks the cheapest known route.
- Use misplaced-tile count as a simpler heuristic, though Manhattan distance is usually more informative.

### Likely viva questions and answers

**Q. What is a state?** A. One complete arrangement of all eight tiles and the blank.

**Q. Why use a tuple?** A. Tuples are immutable and hashable, so they can be dictionary keys.

**Q. Why exclude the blank from Manhattan distance?** A. The blank is not a numbered tile whose destination contributes to this heuristic.

**Q. What is repeated-state detection?** A. Avoiding repeated expansion of a state when it has already been reached at an equal or lower cost.

# Experiment 4: EDA, PCA and Regression

## Q9. Uber Fare Prediction Using PCA and EDA

**Problem focus:** Uber Fare Prediction Using PCA and EDA.

**Dataset / setup:** Kaggle Uber Fares CSV. This code expects `fare_amount`, pickup/dropoff longitude and latitude, `passenger_count`, and optionally `pickup_datetime`.

### Theory to write in the practical record

Exploratory Data Analysis (EDA) is used to understand the dataset through dimensions, data types, summary statistics, missing-value checks and plots. In fare prediction, the fare amount is the target and pickup/dropoff coordinates, passenger count and pickup time are possible features. PCA transforms correlated or high-dimensional standardized features into fewer principal components that retain as much variance as possible. The models with and without PCA are compared using R-squared, RMSE and MAE.

### Beginner-level Python program (with comments)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Download the Uber Fares CSV from Kaggle and place it beside this program.
df = pd.read_csv("uber.csv")
print("Shape:", df.shape)
print("Data types:\n", df.dtypes)
print("Missing values:\n", df.isnull().sum())
print("Summary:\n", df.describe())

# Remove rows with invalid or missing fare values.
df = df.dropna(subset=["fare_amount", "pickup_longitude", "pickup_latitude",
                       "dropoff_longitude", "dropoff_latitude", "passenger_count"])
df = df[(df["fare_amount"] > 0) & (df["fare_amount"] < 500)]
df = df[(df["passenger_count"] > 0) & (df["passenger_count"] <= 6)]

# Convert pickup time into a numeric hour feature, when that column exists.
if "pickup_datetime" in df.columns:
    df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"], errors="coerce")
    df["pickup_hour"] = df["pickup_datetime"].dt.hour

features = ["pickup_longitude", "pickup_latitude", "dropoff_longitude",
            "dropoff_latitude", "passenger_count"]
if "pickup_hour" in df.columns:
    features.append("pickup_hour")
X = df[features].fillna(df[features].median())
y = df["fare_amount"]

# Basic EDA plots.
df["fare_amount"].hist(bins=30)
plt.title("Uber Fare Distribution")
plt.xlabel("Fare amount")
plt.ylabel("Frequency")
plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
# Fit the scaler only on training features to avoid data leakage.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model 1: Linear Regression on the original standardized features.
model_original = LinearRegression()
model_original.fit(X_train_scaled, y_train)
pred_original = model_original.predict(X_test_scaled)

# Model 2: PCA keeps enough components to explain 95% of training variance.
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
model_pca = LinearRegression()
model_pca.fit(X_train_pca, y_train)
pred_pca = model_pca.predict(X_test_pca)

def show_metrics(name, actual, predicted):
    rmse = mean_squared_error(actual, predicted) ** 0.5
    print(name, "R2:", round(r2_score(actual, predicted), 3),
          "RMSE:", round(rmse, 3),
          "MAE:", round(mean_absolute_error(actual, predicted), 3))

show_metrics("Without PCA", y_test, pred_original)
show_metrics("With PCA", y_test, pred_pca)
print("Original feature count:", X_train.shape[1])
print("PCA component count:", pca.n_components_)
print("Explained variance:", pca.explained_variance_ratio_.sum())
```

### Important lines explained

- `read_csv()` loads the Kaggle file; change the filename if yours differs.
- `describe()`, `dtypes` and `isnull().sum()` satisfy the main dataset-inspection tasks.
- The filters remove implausible fare and passenger records; the exact thresholds should be justified and adjusted to the data.
- `pickup_hour` is extracted from the timestamp so a date string is not passed directly to Linear Regression.
- `StandardScaler.fit_transform()` is used on training data; test data uses only `transform()`.
- `PCA(n_components=0.95)` chooses enough components to retain at least 95% of training variance.
- RMSE is the square root of MSE; MAE is the average absolute prediction error; R2 measures explained variance.

### Alternatives and exam-friendly variations

- Use `RobustScaler` when strong outliers remain.
- Use `OneHotEncoder` or cyclical hour features if you add categorical/time features.
- Try Random Forest Regressor or Gradient Boosting as a non-linear alternative, then compare the same metrics.
- PCA is not always beneficial: it may reduce dimensions but also remove information useful for predicting fares.

### Likely viva questions and answers

**Q. What is EDA?** A. The process of inspecting and visualizing data before modeling.

**Q. Why standardize before PCA?** A. PCA is variance-based; features with large numeric scales could dominate without standardization.

**Q. What does explained variance ratio mean?** A. The proportion of total variance represented by a principal component.

**Q. What are RMSE and MAE?** A. Both measure prediction error; RMSE penalizes large errors more strongly, while MAE is the mean absolute error.

## Q10. House Price Prediction Using Linear Regression

**Problem focus:** House Price Prediction Using Linear Regression.

**Dataset / setup:** Housing CSV. The sample expects a target column named `price`; change this one name to match your file. All remaining columns are treated as features.

### Theory to write in the practical record

Linear Regression predicts a continuous target by fitting a linear relationship between input features and the target. For house-price prediction, property characteristics such as area, bedrooms, bathrooms and location-related attributes are inputs, while price is the target. The dataset is inspected, missing data is handled, categorical columns are converted into numerical indicators, and the model is evaluated on records not used for training. R2, RMSE and MAE describe predictive performance.

### Beginner-level Python program (with comments)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Expected file: Housing.csv. Change target name if your dataset uses another name.
df = pd.read_csv("Housing.csv")
print("Shape:", df.shape)
print("Missing values:\n", df.isnull().sum())
print(df.describe(include="all"))

target = "price"
# Separate input columns from the value we want to predict.
X = df.drop(columns=[target])
y = df[target]

# Convert text categories (for example, yes/no or furnishing status) to 0/1 columns.
X = pd.get_dummies(X, drop_first=True, dtype=int)

# Fill missing numeric input values with their column median.
X = X.fillna(X.median())

# Split first so test records are not used to train the prediction model.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("R2:", r2_score(y_test, predictions))
print("RMSE:", mean_squared_error(y_test, predictions) ** 0.5)
print("MAE:", mean_absolute_error(y_test, predictions))

# Compare actual and predicted prices visually.
plt.scatter(y_test, predictions)
plt.xlabel("Actual house price")
plt.ylabel("Predicted house price")
```

```
plt.title("Actual vs Predicted House Prices")
plt.show()
```

### Important lines explained

- `target = "price"` identifies the column to predict. Change it to match the CSV header exactly.
- `X` contains the input features, while `y` contains the target values.
- `pd.get_dummies()` converts categorical text into numeric indicator columns.
- `fillna(X.median())` fills missing numeric values with the median.
- `train_test_split()` keeps a test set for unbiased evaluation.
- `fit()` learns coefficients; `predict()` estimates prices for unseen test rows.
- On the scatter plot, points closer to the diagonal relationship indicate better predictions.

### Alternatives and exam-friendly variations

- For a more rigorous workflow, place `SimpleImputer` and `OneHotEncoder` inside a `ColumnTransformer` and `Pipeline` so preprocessing is fitted only on training data.
- Use Ridge or Lasso Regression to reduce overfitting.
- Use `StandardScaler` when feature scales differ and the chosen model is sensitive to scale; ordinary Linear Regression does not require scaling in the same way as KNN.

### Likely viva questions and answers

**Q. What is the target variable?** A. The variable the model is trained to predict; here it is house price.

**Q. Why use get_dummies?** A. Linear Regression requires numeric inputs, so categories must be encoded.

**Q. What is R2?** A. A score indicating how much variation in the target is explained by the model.

**Q. What does MAE measure?** A. The average absolute difference between actual and predicted prices.

## Q11. Student Performance Prediction Using Regression

**Problem focus:** Student Performance Prediction Using Regression.

**Dataset / setup:** Student performance CSV. This example expects `study_hours`, `attendance`, `previous_marks`, and `final_score`; rename dataset columns as needed.

### Theory to write in the practical record

Student-score prediction is a regression problem when the output is a numerical final score. Possible predictors include study hours, attendance and previous marks. Linear Regression estimates a linear relationship between these features and the final score. The dataset is split into training and testing parts so that performance is measured on unseen student records. Feature coefficients can help interpret which inputs are positively or negatively associated with predicted scores.

### Beginner-level Python program (with comments)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Prepare student_performance.csv with these columns or rename your columns accordingly.
df = pd.read_csv("student_performance.csv")
features = ["study_hours", "attendance", "previous_marks"]
target = "final_score"

X = df[features].copy()
y = df[target]
X = X.fillna(X.median())  # Fill missing input values using column medians.
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("R2:", r2_score(y_test, predictions))
print("RMSE:", mean_squared_error(y_test, predictions) ** 0.5)
print("MAE:", mean_absolute_error(y_test, predictions))

# Display each feature's learned coefficient.
for feature, coefficient in zip(features, model.coef_):
    print(feature, "coefficient:", round(coefficient, 3))
print("Intercept:", round(model.intercept_, 3))
```

### Important lines explained

- The feature list contains the selected academic inputs; update it to match the dataset.
- `final_score` is the numerical target, so this is regression rather than classification.
- `median()` is used to fill missing numerical inputs.
- `model.coef_` gives one learned coefficient per input feature.
- A positive coefficient means the predicted score increases as that feature increases, keeping other features fixed; a negative coefficient means it decreases.
- A coefficient is an association learned by the model, not proof of causation.

### Alternatives and exam-friendly variations

- Use `G1`, `G2`, `studytime` and `absences` if you use the UCI student-mat/student-por dataset; set `G3` as target.
- Use Random Forest Regressor to model non-linear relationships.
- Use MAE when you want an error measure in the same units as the final score.

### Likely viva questions and answers

**Q. Why is this regression?** A. Because the target final score is a numerical value.

**Q. What does a positive coefficient mean?** A. The model predicts a higher score as that feature increases, holding the other features fixed.

**Q. Why split the data?** A. To evaluate performance on data the model did not train on.

**Q. Does correlation prove causation?** A. No. A predictive association does not establish a causal effect.

## Q12. Salary Estimation Using Linear Regression

**Problem focus:** Salary Estimation Using Linear Regression.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Salary estimation from years of experience is a simple supervised regression problem. The input feature is years of experience and the target is salary. Linear Regression fits a straight line y = mx + c, where m is the slope and c is the intercept. After training, the model can estimate salary for a user-entered experience value. R2, RMSE and MAE are used to evaluate the predictions.

### Beginner-level Python program (with comments)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Small example dataset; replace these values with your provided salary data.
experience = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
salary = np.array([25000, 30000, 35000, 40000, 48000,
                   52000, 60000, 65000, 72000, 80000])

X_train, X_test, y_train, y_test = train_test_split(
   experience, salary, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Regression equation: Salary =",
    round(model.coef_[0], 2), "* Experience +", round(model.intercept_, 2))
print("R2:", r2_score(y_test, predictions))
print("RMSE:", mean_squared_error(y_test, predictions) ** 0.5)
print("MAE:", mean_absolute_error(y_test, predictions))

# Plot all example observations and the fitted regression line.
line_x = np.linspace(experience.min(), experience.max(), 100).reshape(-1, 1)
line_y = model.predict(line_x)
plt.scatter(experience, salary, label="Actual data")
plt.plot(line_x, line_y, label="Regression line")
plt.xlabel("Years of experience")
plt.ylabel("Salary")
plt.legend()
plt.show()

years = float(input("Enter years of experience: "))
predicted_salary = model.predict([[years]])[0]
print("Estimated salary:", round(predicted_salary, 2))
```

### Important lines explained

- `reshape(-1, 1)` converts the experience list into a two-dimensional feature matrix expected by scikit-learn.
- `train_test_split()` reserves test observations.
- `coef_[0]` is the slope m; `intercept_` is c.
- `np.linspace()` makes many x values so the fitted line looks smooth.
- `predict([[years]])` predicts salary for one new record; the nested list keeps the input two-dimensional.

### Alternatives and exam-friendly variations

- Use `pd.read_csv()` when a salary CSV is provided instead of manually creating arrays.
- Use Polynomial Regression if the relationship is clearly curved, but begin with a straight line for this practical.
- Use `model.score(X_test, y_test)` as a quick way to obtain R2.

### Likely viva questions and answers

**Q. What is the equation of Linear Regression?** A. y = mx + c.

**Q. What is slope?** A. The estimated change in salary for a one-unit increase in experience.

**Q. What is intercept?** A. The model's predicted salary when experience is zero.

**Q. Why reshape the input?** A. scikit-learn expects X to have rows as samples and columns as features.

## Q13. Compare Regression Models With and Without PCA

**Problem focus:** Compare Regression Models With and Without PCA.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Dimensionality reduction transforms many original features into a smaller set of components. PCA creates orthogonal components that capture decreasing amounts of variance. To compare fairly, the same train-test split and same regression algorithm should be used for both models. Scaling and PCA must be fitted only on the training data to avoid data leakage. The comparison considers R2, RMSE and MAE as well as the number of resulting features.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# The built-in Diabetes dataset is numerical and is available with scikit-learn.
data = load_diabetes()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model A: original features (no PCA).
scaler_a = StandardScaler()
X_train_a = scaler_a.fit_transform(X_train)
X_test_a = scaler_a.transform(X_test)
model_a = LinearRegression()
model_a.fit(X_train_a, y_train)
pred_a = model_a.predict(X_test_a)

# Model B: standardized features transformed by PCA.
scaler_b = StandardScaler()
X_train_scaled = scaler_b.fit_transform(X_train)
X_test_scaled = scaler_b.transform(X_test)
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
model_b = LinearRegression()
model_b.fit(X_train_pca, y_train)
pred_b = model_b.predict(X_test_pca)

def evaluate(name, actual, predicted):
    print(name)
    print("R2:", round(r2_score(actual, predicted), 3))
    print("RMSE:", round(mean_squared_error(actual, predicted) ** 0.5, 3))
    print("MAE:", round(mean_absolute_error(actual, predicted), 3))

evaluate("Without PCA", y_test, pred_a)
evaluate("With PCA", y_test, pred_b)
print("Features before PCA:", X_train.shape[1])
print("Components after PCA:", pca.n_components_)
```

### Important lines explained

- The built-in Diabetes dataset avoids the need to download a CSV for this comparison.
- Both models use the same split and Linear Regression, so PCA is the main changed factor.
- `StandardScaler` is fitted on training data only.
- `PCA.fit_transform()` learns components from training data; `transform()` applies them to test data.
- `n_components=0.95` retains at least 95% of training variance.
- Compare the metrics and the number of components; fewer components do not automatically mean better predictions.

### Alternatives and exam-friendly variations

- Choose a fixed number of components, for example `PCA(n_components=5)`, when the practical asks for a specific component count.
- Use `PCA(n_components=None)` to inspect the full explained-variance curve.
- Use Ridge Regression instead of Linear Regression if multicollinearity or overfitting is a concern.

### Likely viva questions and answers

**Q. What is dimensionality reduction?** A. Reducing the number of input features while retaining useful information.

**Q. Are PCA components original features?** A. No. They are new combinations of the original standardized features.

**Q. Why fit PCA only on training data?** A. To prevent test data from influencing the learned transformation.

**Q. Does PCA always improve accuracy?** A. No. It may remove noise but can also discard information relevant to the target.

# Experiment 5: Decision Tree, KNN and Naive Bayes

## Q14. Loan Approval Prediction Using Decision Tree

**Problem focus:** Loan Approval Prediction Using Decision Tree.

**Dataset / setup:** Loan approval CSV. Expected target is `Loan_Status`; code optionally drops `Loan_ID`. Change names to match the provided dataset.

### Theory to write in the practical record

A Decision Tree classifier predicts a class by repeatedly splitting data using feature-based rules. At each node it selects a split that makes the resulting groups purer. Gini impurity and entropy are common splitting criteria. Loan approval is the target class, while applicant characteristics are input features. Accuracy, precision, recall, F1-score and a confusion matrix help evaluate the predictions.

### Beginner-level Python program (with comments)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Expected file: loan_data.csv; target column: Loan_Status (Y/N or 1/0).
df = pd.read_csv("loan_data.csv")
target = "Loan_Status"
if "Loan_ID" in df.columns:
    df = df.drop(columns=["Loan_ID"])  # Identifier is not a useful predictor.

# Convert approval labels to numbers.
y = df[target].replace({"Y": 1, "N": 0, "Yes": 1, "No": 0})
X = df.drop(columns=[target])
X = pd.get_dummies(X, drop_first=True, dtype=int)
X = X.fillna(X.median())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions, zero_division=0))
print("Recall:", recall_score(y_test, predictions, zero_division=0))
print("F1-score:", f1_score(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))

plt.figure(figsize=(14, 7))
plot_tree(model, feature_names=X.columns, class_names=["Rejected", "Approved"],
          filled=True, rounded=True)
plt.show()
```

### Important lines explained

- The target column contains the approval decision; the remaining columns are applicant features.
- `Loan_ID` is dropped because it identifies a record rather than describing an applicant.
- `replace()` converts common yes/no target labels to numeric classes.
- `get_dummies()` encodes categorical inputs.
- `criterion="gini"` selects Gini impurity; `max_depth=4` limits tree depth to reduce overfitting.

- `plot_tree()` displays the learned decision rules.
- Precision, recall and F1-score are calculated for the positive class (approved = 1).

### Alternatives and exam-friendly variations

- Set `criterion="entropy"` to use information gain.
- Try `max_depth=None` for a fully grown tree, then compare whether test performance worsens.
- For a safer preprocessing workflow, use a `Pipeline` with imputers and encoders fitted on the training data only.

### Likely viva questions and answers

**Q. What is a Decision Tree?** A. A tree-shaped model that predicts by applying feature-based decisions at internal nodes.

**Q. What is Gini impurity?** A. A measure of how mixed the classes are at a node; lower impurity means a purer node.

**Q. What is overfitting?** A. When a model learns training-specific patterns and performs poorly on new data.

**Q. Why use a confusion matrix?** A. It shows counts of correct and incorrect predictions for each class.

## Q15. Customer Churn Prediction Using Decision Tree

**Problem focus:** Customer Churn Prediction Using Decision Tree.

**Dataset / setup:** Telco churn CSV, such as the common IBM Telco Customer Churn dataset. Expected target column: `Churn` with Yes/No values.

### Theory to write in the practical record

Customer churn prediction is a binary classification problem: a customer either leaves the service (churn) or stays. A Decision Tree learns rules from customer attributes and account information. Data preparation includes converting numeric-looking text fields, encoding categories and handling missing values. Evaluation uses a confusion matrix, precision, recall and F1-score; recall for the churn class can be important when missed churners are costly.

### Beginner-level Python program (with comments)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score, accuracy_score

# Expected Telco churn file: telco_churn.csv, with target column Churn.
df = pd.read_csv("telco_churn.csv")
if "customerID" in df.columns:
    df = df.drop(columns=["customerID"])

# TotalCharges may be stored as text even though it represents a number.
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

y = df["Churn"].map({"Yes": 1, "No": 0})
X = df.drop(columns=["Churn"])
X = pd.get_dummies(X, drop_first=True, dtype=int)
X = X.fillna(X.median())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
print("Precision:", precision_score(y_test, predictions, zero_division=0))
print("Recall:", recall_score(y_test, predictions, zero_division=0))
print("F1-score:", f1_score(y_test, predictions, zero_division=0))
```

### Important lines explained

- `customerID` is an identifier, so it is removed.
- `pd.to_numeric(..., errors="coerce")` converts `TotalCharges` to numbers and changes invalid text to missing values.
- `map()` changes Yes/No churn labels into 1/0.
- `get_dummies()` converts categorical features to numeric columns.
- `stratify=y` tries to preserve the churn/non-churn proportions in train and test sets.
- `max_depth=5` limits complexity.
- The confusion matrix shows true negatives, false positives, false negatives and true positives in a binary problem.

### Alternatives and exam-friendly variations

- Use Random Forest for an ensemble of decision trees.
- Use class weights such as `class_weight="balanced"` when churn classes are imbalanced.
- Use ROC-AUC or precision-recall curves for threshold-sensitive analysis.

### Likely viva questions and answers

**Q. What is churn?** A. When a customer stops using or paying for a service.

**Q. What does stratify do?** A. It preserves approximately the same class proportions in each split.

**Q. Why is recall important in churn?** A. Low recall means many customers who will leave are missed.

**Q. What does max_depth control?** A. The maximum number of decision levels in the tree.

## Q16. Disease Classification Using Decision Tree

**Problem focus:** Disease Classification Using Decision Tree.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Disease classification is a supervised classification problem when each medical record has a known diagnostic label. A Decision Tree learns rules from numerical measurements and predicts a diagnostic class for a new record. The scikit-learn Breast Cancer Wisconsin dataset is a labeled public dataset with numerical features and two classes. The model is evaluated using accuracy, precision, recall, F1-score and a confusion matrix. This is an educational classification exercise, not a medical diagnostic tool.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Load a built-in, labelled dataset (no external CSV is required).
data = load_breast_cancer()
X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Classes:", data.target_names)
print("Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions, zero_division=0))
print("Recall:", recall_score(y_test, predictions, zero_division=0))
print("F1-score:", f1_score(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- `load_breast_cancer()` returns a dataset object containing features, labels and class names.
- `data.data` contains numerical input measurements; `data.target` contains diagnostic class labels.
- `stratify=y` preserves class proportions in the split.
- `DecisionTreeClassifier` learns the classification rules.
- `target_names` shows which numeric label represents each class.
- The confusion matrix and metrics evaluate predictions against known test labels.

### Alternatives and exam-friendly variations

- Use the Iris dataset for a simpler multi-class example.
- Use Random Forest for a potentially more stable ensemble model.
- Use `class_weight="balanced"` if the diagnostic classes are imbalanced.

### Likely viva questions and answers

**Q. What is supervised learning?** A. Learning a mapping from input examples that have known labels.

**Q. What is classification?** A. Predicting a discrete category or class.

**Q. What is the target?** A. The known diagnostic class stored in `data.target`.

**Q. Can this model be used for real diagnosis?** A. No. It is an educational example and is not a clinically validated diagnostic system.

## Q17. Decision Tree Optimization: Entropy vs Gini

**Problem focus:** Decision Tree Optimization: Entropy vs Gini.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

A Decision Tree can use different criteria to choose a split. Gini impurity measures the chance of incorrect class assignment from the node's class distribution. Entropy measures uncertainty, and information gain measures the reduction in entropy after a split. Hyperparameters such as maximum depth and minimum samples per leaf control tree complexity. Cross-validation evaluates parameter choices over multiple training/validation folds.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load Iris as a simple multi-class dataset.
data = load_iris()
```

```
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

# Train and compare one Gini tree and one Entropy tree.
gini_tree = DecisionTreeClassifier(criterion="gini", random_state=42)
entropy_tree = DecisionTreeClassifier(criterion="entropy", random_state=42)
gini_tree.fit(X_train, y_train)
entropy_tree.fit(X_train, y_train)
print("Gini accuracy:", accuracy_score(y_test, gini_tree.predict(X_test)))
print("Entropy accuracy:", accuracy_score(y_test, entropy_tree.predict(X_test)))
print("Gini depth/leaves:", gini_tree.get_depth(), gini_tree.get_n_leaves())
print("Entropy depth/leaves:", entropy_tree.get_depth(), entropy_tree.get_n_leaves())

# Try several values and select the best using 5-fold cross-validation.
parameter_grid = {
   "criterion": ["gini", "entropy"],
   "max_depth": [2, 3, 4, None],
   "min_samples_leaf": [1, 2, 4]
}
search = GridSearchCV(DecisionTreeClassifier(random_state=42),
                      parameter_grid, cv=5, scoring="accuracy")
search.fit(X_train, y_train)
best_model = search.best_estimator_
print("Best parameters:", search.best_params_)
print("Best cross-validation score:", search.best_score_)
print("Tuned test accuracy:", accuracy_score(y_test, best_model.predict(X_test)))
```

### Important lines explained

- Both initial trees use the same training and testing split; only the criterion changes.
- `get_depth()` and `get_n_leaves()` describe the learned tree structure.
- `parameter_grid` lists the candidate hyperparameter values.
- `GridSearchCV` trains models for combinations and evaluates them using five folds.
- `best_estimator_` is the model with the best average cross-validation score.
- The final test score is calculated only after tuning on the training data.

### Alternatives and exam-friendly variations

- Use `RandomizedSearchCV` when the parameter space is large.
- Add `class_weight` or `ccp_alpha` to the search grid for imbalanced classes or pruning.
- Use `scoring="f1_macro"` for an imbalanced multi-class task instead of accuracy.

### Likely viva questions and answers

**Q. Difference between Gini and Entropy?** A. Both measure impurity; Gini uses squared class probabilities, while entropy uses information content.

**Q. What is max_depth?** A. The maximum depth of the decision tree.

**Q. What is cross-validation?** A. Repeated training and validation on different folds of the training set.

**Q. Why keep a final test set?** A. To estimate performance on data not used for fitting or hyperparameter selection.

## Q18. Iris Flower Classification Using KNN

**Problem focus:** Iris Flower Classification Using KNN.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

K-Nearest Neighbors (KNN) is a supervised algorithm that classifies a new sample based on the labels of its nearest training samples. The parameter K specifies how many neighbours vote. Since KNN uses distances, feature scaling is important when measurement scales differ. The Iris dataset has four numerical flower measurements and three species labels. Testing several K values helps select a suitable neighbour count.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load flower measurements and species labels.
data = load_iris()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Learn mean and standard deviation from training data only.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Compare three neighbour counts.
for k in [3, 5, 7]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)
    print("K =", k, "Accuracy =", accuracy_score(y_test, predictions))

# Example prediction for a new flower (measurements in the Iris feature order).
new_flower = [[5.1, 3.5, 1.4, 0.2]]
new_flower_scaled = scaler.transform(new_flower)
chosen_model = KNeighborsClassifier(n_neighbors=5)
chosen_model.fit(X_train_scaled, y_train)
print("Predicted species:", data.target_names[chosen_model.predict(new_flower_scaled)[0]])
```

### Important lines explained

- `load_iris()` supplies four measurements and an integer species label.
- `StandardScaler` transforms each feature to a comparable scale.
- `fit_transform()` is used on the training data; the test data uses the same training statistics through `transform()`.
- `n_neighbors=k` sets the number of neighbours voting on the class.
- The loop compares K=3, K=5 and K=7 using the same split.
- The new flower is scaled before prediction because the model was trained on scaled features.

### Alternatives and exam-friendly variations

- Use Manhattan distance with `metric="manhattan"` instead of the default Euclidean distance.
- Use odd K values in a binary classification problem to reduce tied votes; for multi-class data ties can still occur.
- Try a range of K values and plot accuracy against K.

### Likely viva questions and answers

**Q. What is K in KNN?** A. The number of nearest training examples used to make a prediction.

**Q. Why scale KNN features?** A. Because the algorithm uses distances, large-scale features could dominate.

**Q. Is KNN supervised or unsupervised?** A. Supervised, because it learns from labelled training examples.

**Q. What is the prediction rule?** A. Usually majority voting among the K nearest neighbours.

## Q19. Student Performance Classification Using KNN

**Problem focus:** Student Performance Classification Using KNN.

**Dataset / setup:** Student performance CSV. Expected columns: `attendance`, `study_hours`, `internal_marks`, and `final_score`; change feature names if needed.

### Theory to write in the practical record

Student performance classification predicts a category such as Low, Average or High rather than an exact numerical score. Relevant inputs may include attendance, study hours and internal marks. KNN classifies a student by comparing feature distances to labelled training students. The features should be scaled, and the K value should be compared using a fixed train-test split. The categories are a simplified grouping for this exercise, not a definitive measure of student ability.

### Beginner-level Python program (with comments)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix

# Expected columns: attendance, study_hours, internal_marks, final_score.
df = pd.read_csv("student_performance.csv")
features = ["attendance", "study_hours", "internal_marks"]
X = df[features].fillna(df[features].median())

# Create simple target categories from final score if no category is supplied.
y = pd.cut(df["final_score"], bins=[-1, 40, 70, 100],
        labels=["Low", "Average", "High"])

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

for k in [3, 5, 7]:
   model = KNeighborsClassifier(n_neighbors=k)
   model.fit(X_train_scaled, y_train)
   predictions = model.predict(X_test_scaled)
   print("K =", k, "Accuracy =", accuracy_score(y_test, predictions))

# Show classification results for K=5.
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_scaled, y_train)
predictions = model.predict(X_test_scaled)
print("Precision (macro):", precision_score(y_test, predictions, average="macro", zero_division=0))
print("Recall (macro):", recall_score(y_test, predictions, average="macro", zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- The program expects three input features and a numeric final score.
- `pd.cut()` converts the score into three labelled categories. The interval boundaries can be changed to match your practical definition.

- `stratify=y` helps preserve the category distribution in both sets.
- `StandardScaler` is important because attendance, study hours and marks use different scales.
- `average="macro"` calculates each class metric and then gives all classes equal weight.

### Alternatives and exam-friendly variations

- If the CSV already has a `performance` category column, use `y = df["performance"]` instead of `pd.cut()`.
- Try K values from 1 to 15 and select using validation accuracy.
- Use weighted or macro F1-score when class sizes are unequal.

### Likely viva questions and answers

**Q. Why is this classification?** A. The target is a category (Low, Average or High), not a continuous score.

**Q. Why use scaling?** A. KNN relies on feature distances.

**Q. What does macro average mean?** A. It gives equal importance to each class when averaging the metric.

**Q. Do clusters/categories prove ability?** A. No. They are simplified labels for this dataset and should not be treated as definitive ability measures.

## Q20. Diabetes Classification Using KNN

**Problem focus:** Diabetes Classification Using KNN.

**Dataset / setup:** Pima Indians Diabetes CSV. Expected target `Outcome` and columns such as Glucose, BloodPressure, SkinThickness, Insulin and BMI.

### Theory to write in the practical record

The Pima Indians Diabetes dataset is a binary classification dataset with medical measurements and an outcome label. KNN predicts the outcome by comparing a record with nearby training records. Some columns use zero to represent missing or invalid measurements, for example glucose, blood pressure, skin thickness, insulin and BMI. These zeros should be handled as missing values before scaling. Stratified splitting preserves the outcome proportions.

### Beginner-level Python program (with comments)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix, accuracy_score

# Expected Pima file: diabetes.csv, target column Outcome (0/1).
df = pd.read_csv("diabetes.csv")
invalid_zero_columns = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
for column in invalid_zero_columns:
    if column in df.columns:
        df[column] = df[column].replace(0, float("nan"))

X = df.drop(columns=["Outcome"])
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Imputer and scaler are fitted only on the training data.
imputer = SimpleImputer(strategy="median")
X_train_clean = imputer.fit_transform(X_train)
X_test_clean = imputer.transform(X_test)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_clean)
```

```
X_test_scaled = scaler.transform(X_test_clean)

for k in [3, 5, 7, 9]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)
    print("K =", k, "Accuracy =", accuracy_score(y_test, predictions),
          "Precision =", precision_score(y_test, predictions, zero_division=0),
          "Recall =", recall_score(y_test, predictions, zero_division=0),
          "F1 =", f1_score(y_test, predictions, zero_division=0))

print("Confusion matrix for last K:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- The invalid-zero list contains measurements for which zero is generally not a valid real measurement in this dataset.
- `replace(0, pd.NA)` marks those values as missing.
- The split happens before imputation and scaling so test data does not affect fitted preprocessing.
- `SimpleImputer(strategy="median")` replaces missing values with the training median.
- KNN is trained and evaluated for several neighbour counts.
- Precision, recall and F1-score focus on positive-class detection.

### Alternatives and exam-friendly variations

- Use a `Pipeline` to combine imputation, scaling and KNN in one object.
- Try `weights="distance"` so closer neighbours have more influence.
- Use cross-validation to select K instead of comparing only one test split.

### Likely viva questions and answers

**Q. Why replace some zeros?** A. In the standard Pima dataset, zero values in certain measurements represent missing/invalid entries.

**Q. Why stratify?** A. To preserve the proportion of diabetic and non-diabetic examples.

**Q. Why impute before KNN?** A. Distance-based calculations cannot safely use missing values.

**Q. Why scale after imputation?** A. To put numerical measurements on comparable ranges.

## Q21. Email or SMS Spam Detection Using Naive Bayes

**Problem focus:** Email or SMS Spam Detection Using Naive Bayes.

**Dataset / setup:** Spam CSV such as the SMS Spam Collection. Expected columns `label` and `message`, or common `v1` and `v2` columns.

### Theory to write in the practical record

Spam detection is a text-classification task. A labelled dataset contains message text and a class such as spam or ham. Text must be converted into numerical features before a classifier can learn from it. CountVectorizer represents word counts, while TF-IDF weights words by their importance across messages. Multinomial Naive Bayes is commonly used for word-count or TF-IDF features. The vectorizer must be fitted on training text only to avoid data leakage.

### Beginner-level Python program (with comments)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
```

```
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix, accuracy_score

# Expected file spam.csv with columns: label (spam/ham) and message.
df = pd.read_csv("spam.csv", encoding="latin-1")
# Common SMS Spam Collection files use v1 and v2; rename them if present.
if "v1" in df.columns and "v2" in df.columns:
    df = df.rename(columns={"v1": "label", "v2": "message"})

df = df[["label", "message"]].dropna()
X = df["message"].astype(str)
y = df["label"].map({"ham": 0, "spam": 1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Pipeline ensures TF-IDF is fitted only on the training text.
model = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english")),
    ("naive_bayes", MultinomialNB())
])
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions, zero_division=0))
print("Recall:", recall_score(y_test, predictions, zero_division=0))
print("F1-score:", f1_score(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))

new_message = [input("Enter a message to classify: ")]
result = model.predict(new_message)[0]
print("Spam" if result == 1 else "Not spam")
```

### Important lines explained

- The code supports both `label/message` and the common `v1/v2` SMS file headers.
- `astype(str)` ensures all messages are text.
- `map()` changes ham/spam into numeric labels.
- `TfidfVectorizer` converts text into weighted word features and removes common English stop words.
- `Pipeline` ensures vectorizer fitting occurs only on training text.
- `MultinomialNB` learns class probabilities from the word features.
- The final input lets you test a new message after evaluation.

### Alternatives and exam-friendly variations

- Use `CountVectorizer()` instead of `TfidfVectorizer()` to use raw word counts.
- Use `stop_words="english"` only when English stop-word removal is appropriate for the language of the messages.
- For short SMS messages, character n-grams can help detect obfuscated spam words.

### Likely viva questions and answers

**Q. What is Naive Bayes?** A. A probabilistic classifier based on Bayes theorem and a conditional-independence assumption.

**Q. What is TF-IDF?** A. A weighting method that gives more weight to words that are frequent in a document but less common across documents.

**Q. Why use MultinomialNB?** A. It is suitable for non-negative word-count or TF-IDF feature values.

**Q. What is ham?** A. A legitimate, non-spam message.

## Q22. Iris Classification Using Gaussian Naive Bayes vs Decision Tree

**Problem focus:** Iris Classification Using Gaussian Naive Bayes vs Decision Tree.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

Gaussian Naive Bayes applies Bayes theorem and assumes that each feature follows a Gaussian distribution within each class. It also makes a conditional-independence assumption: features are treated as independent given the class. A Decision Tree instead learns feature-based splitting rules. Both models should use the same training and testing split so their evaluation is comparable. Accuracy, precision, recall, F1-score and confusion matrices are used for comparison.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Load the same Iris dataset for both classifiers.
data = load_iris()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

models = {
    "Gaussian Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, predictions))
    print("Precision (macro):", precision_score(y_test, predictions, average="macro"))
    print("Recall (macro):", recall_score(y_test, predictions, average="macro"))
    print("F1 (macro):", f1_score(y_test, predictions, average="macro"))
    print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- The same `X_train`, `X_test`, `y_train` and `y_test` are used for both models.
- `GaussianNB()` is the Gaussian Naive Bayes classifier for continuous numerical features.
- `DecisionTreeClassifier()` learns split rules from the same features.
- `average="macro"` treats the three Iris classes equally.
- Each confusion matrix shows which species were correctly or incorrectly classified.

### Alternatives and exam-friendly variations

- Use `MultinomialNB` for word-count or non-negative count features.
- Use `BernoulliNB` for binary features.
- Use a fixed `random_state` so the train-test split is repeatable.

### Likely viva questions and answers

**Q. What is Bayes theorem?** A. $P(A|B) = P(B|A)P(A)/P(B)$, when $P(B)$ is non-zero.

**Q. What is the Naive assumption?** A. Features are conditionally independent given the class.

**Q. Why Gaussian Naive Bayes?** A. It models continuous features using a Gaussian distribution per class.

**Q. Why use the same split?** A. So the comparison is fair and is not affected by different test records.

# Experiment 6: Ensemble Learning and K-Means Clustering

## Q23. Compare Random Forest and Decision Tree

**Problem focus:** Compare Random Forest and Decision Tree.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

A Decision Tree is a single tree that learns prediction rules from data. A Random Forest is an ensemble of many decision trees trained on bootstrap samples and random subsets of features. Their predictions are combined, usually by majority voting for classification. Random Forest often reduces the variance of a single tree, although it can be less interpretable and more computationally expensive. Both models are compared on the same test records.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score

# Load the Iris classification dataset.
data = load_iris()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, predictions))
    print("Precision (macro):", precision_score(y_test, predictions, average="macro"))
    print("Recall (macro):", recall_score(y_test, predictions, average="macro"))
    print("F1-score (macro):", f1_score(y_test, predictions, average="macro"))
```

### Important lines explained

- A common train-test split keeps the comparison fair.
- `DecisionTreeClassifier()` trains one decision tree.
- `RandomForestClassifier(n_estimators=100)` trains 100 trees and combines their predictions.
- `random_state` makes the experiment reproducible.
- Macro metrics give every Iris class equal weight.

### Alternatives and exam-friendly variations

- Change `n_estimators` to 50 or 200 to compare forest sizes.
- Set `max_depth` to limit the depth of each tree.
- Use out-of-bag scoring with `oob_score=True` when `bootstrap=True` to estimate generalization without a separate validation split.

### Likely viva questions and answers

**Q. What is ensemble learning?** A. Combining multiple models to obtain a stronger or more stable prediction.

**Q. What is Random Forest?** A. An ensemble of decision trees trained with bootstrap samples and random feature subsets.

**Q. What does n_estimators mean?** A. The number of trees in the forest.

**Q. Main advantage over one tree?** A. It often reduces variance and overfitting compared with a single deep tree.

## Q24. Titanic Survival Prediction Using Random Forest

**Problem focus:** Titanic Survival Prediction Using Random Forest.

**Dataset / setup:** Titanic CSV. Expected columns: `Survived`, `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`, and `Embarked`.

### Theory to write in the practical record

Titanic survival prediction is a binary classification problem. The target is whether a passenger survived, while useful inputs include passenger class, sex, age, fare and family-related counts. Missing age and embarkation values must be handled, and categorical features such as sex and embarkation port must be encoded. Random Forest combines multiple decision trees and can model non-linear feature relationships. This practical evaluates accuracy, precision, recall, F1-score and the confusion matrix.

### Beginner-level Python program (with comments)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Expected Titanic CSV columns include Survived, Pclass, Sex, Age, SibSp, Parch, Fare, Embarked.
df = pd.read_csv("titanic.csv")
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[features].copy()
y = df["Survived"]

# Fill missing numerical and categorical values.
X["Age"] = X["Age"].fillna(X["Age"].median())
X["Fare"] = X["Fare"].fillna(X["Fare"].median())
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])

# Convert Sex and Embarked categories into numeric columns.
X = pd.get_dummies(X, columns=["Sex", "Embarked"], drop_first=True, dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions, zero_division=0))
print("Recall:", recall_score(y_test, predictions, zero_division=0))
print("F1-score:", f1_score(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- The feature list selects only the passenger fields used in this example.
- `median()` fills missing numerical values; `mode()[0]` fills the most common embarkation category.

- `get_dummies()` converts Sex and Embarked into numeric columns.
- `stratify=y` preserves the survival class balance in the split.
- `n_estimators=100` trains 100 trees.
- The metrics compare predicted survival labels with the known test labels.

### Alternatives and exam-friendly variations

- Add `Ticket` or `Cabin` only after considering how to encode them and how much missing data they contain.
- Use a `ColumnTransformer` and `Pipeline` for a more rigorous preprocessing workflow.
- Tune `max_depth`, `min_samples_leaf` and `max_features` using cross-validation.

### Likely viva questions and answers

**Q. What is the target column?** A. `Survived`, commonly 0 for did not survive and 1 for survived.

**Q. Why encode Sex and Embarked?** A. Machine-learning estimators require numerical feature values.

**Q. What does Random Forest combine?** A. Predictions from multiple decision trees.

**Q. Why use stratification?** A. To keep the survival/non-survival proportions similar in training and testing data.

## Q25. Customer Segmentation Using K-Means

**Problem focus:** Customer Segmentation Using K-Means.

**Dataset / setup:** Mall Customers CSV. Expected headers: `Annual Income (k$)` and `Spending Score (1-100)`.

### Theory to write in the practical record

K-Means is an unsupervised clustering algorithm that divides observations into K groups. It assigns each point to the nearest centroid and repeatedly updates each centroid to the mean of its assigned points. For customer segmentation, annual income and spending score can be used as the two numerical features. The elbow method plots within-cluster sum of squares (inertia) for different K values; the bend in the curve is a candidate cluster count. Feature scaling is important when features use different units or ranges.

### Beginner-level Python program (with comments)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Expected file: Mall_Customers.csv.
df = pd.read_csv("Mall_Customers.csv")
features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].dropna()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow method: compare inertia for K from 1 to 10.
inertias = []
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)

plt.plot(range(1, 11), inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
```

```
plt.title("Elbow Method")
plt.show()

# Choose K after inspecting the elbow plot; 5 is a common demonstration value.
chosen_k = 5
kmeans = KMeans(n_clusters=chosen_k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# Plot clusters in the original two-feature scale.
plt.scatter(X[features[0]], X[features[1]], c=labels)
centres = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(centres[:, 0], centres[:, 1], marker="X", s=200, label="Centroids")
plt.xlabel(features[0])
plt.ylabel(features[1])
plt.legend()
plt.title("Customer Segments")
plt.show()

for cluster in range(chosen_k):
    print("Cluster", cluster, "customers:", sum(labels == cluster))
```

### Important lines explained

- `X` selects annual income and spending score, the two features requested by the problem.
- `StandardScaler` puts the features on comparable scales.
- `inertia_` is the within-cluster sum of squared distances to centroids.
- The elbow plot helps identify a K beyond which reducing inertia gives diminishing returns.
- `fit_predict()` both fits K-Means and returns a cluster number for each customer.
- `inverse_transform()` converts centroid coordinates back to the original income/spending-score scale.

### Alternatives and exam-friendly variations

- Use `silhouette_score` to supplement the elbow method; do not rely on one plot alone.
- Use `MiniBatchKMeans` for very large datasets.
- Use other customer features only when they are available, meaningful and appropriately scaled.

### Likely viva questions and answers

**Q. Is K-Means supervised?** A. No. It is unsupervised because it does not require class labels.

**Q. What is a centroid?** A. The mean position of the points assigned to a cluster.

**Q. What is inertia?** A. The sum of squared distances from points to their assigned centroids.

**Q. What is the elbow method?** A. A visual method for choosing K by looking for a bend in the inertia curve.

## Q26. Student Grouping Using K-Means

**Problem focus:** Student Grouping Using K-Means.

**Dataset / setup:** Student performance CSV. Expected numerical columns: `attendance`, `internal_marks`, and `assignment_score`.

### Theory to write in the practical record

Student grouping with K-Means is an unsupervised learning task. Each student is represented by numerical attributes such as attendance, internal marks and assignment score. Since these features have different scales, standardization is applied before clustering. The elbow method suggests a suitable number of clusters. When three or more features are used, PCA can project the data into two dimensions for plotting. Clusters describe patterns in this dataset and should not be treated as fixed measures of ability.

### Beginner-level Python program (with comments)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Expected columns: attendance, internal_marks, assignment_score.
df = pd.read_csv("student_performance.csv")
features = ["attendance", "internal_marks", "assignment_score"]
X = df[features].dropna()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Use the elbow method to compare candidate K values.
inertias = []
for k in range(1, 8):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)
plt.plot(range(1, 8), inertias, marker="o")
plt.xlabel("K")
plt.ylabel("Inertia")
plt.title("Elbow Method for Student Groups")
plt.show()

chosen_k = 3  # Adjust this after examining the elbow plot.
kmeans = KMeans(n_clusters=chosen_k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# PCA converts the three standardized inputs into two plotting coordinates.
pca = PCA(n_components=2)
points_2d = pca.fit_transform(X_scaled)
plt.scatter(points_2d[:, 0], points_2d[:, 1], c=labels)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Student Clusters (PCA Projection)")
plt.show()

# Print the average original feature values in each group.
result = X.copy()
result["Cluster"] = labels
print(result.groupby("Cluster")[features].mean())
```

### Important lines explained

- Three selected numerical columns form the feature matrix.
- `StandardScaler` prevents a feature with a larger numeric range from dominating the distance.
- The elbow loop fits K-Means for several candidate cluster counts.
- `chosen_k` is selected after looking at the elbow plot; 3 is only a starting example.
- `PCA(n_components=2)` creates two coordinates so three-dimensional data can be plotted.
- `groupby().mean()` gives a simple profile of each student cluster.

### Alternatives and exam-friendly variations

- Use `silhouette_score` to assess cluster separation.
- Use `PCA(n_components=2)` for visualization only; the clustering in this example uses all three standardized features.
- Try different random seeds to check whether cluster assignments are stable.

### Likely viva questions and answers

**Q. Why standardize student features?** A. Because K-Means uses distances and features can have different ranges.

**Q. Why PCA?** A. It projects multiple features into two dimensions for visualization.

**Q. Are clusters labelled pass/fail?** A. No. K-Means creates groups without given target labels.

**Q. Can a cluster label measure a student's ability definitively?** A. No. It is only a pattern in the selected dataset and features.

## Q27. Mall Customer Clustering

**Problem focus:** Mall Customer Clustering.

**Dataset / setup:** Mall Customers CSV. Expected headers: `Annual Income (k$)` and `Spending Score (1-100)`.

### Theory to write in the practical record

Mall customer clustering uses the Mall Customers dataset, which commonly includes CustomerID, Gender, Age, Annual Income and Spending Score. The problem specifies annual income and spending score as the clustering features. K-Means groups customers by similarity in those two features. Standardization and the elbow method are used before selecting K. The final plot displays the clusters and centroids, and the number of customers in each cluster is reported.

### Beginner-level Python program (with comments)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Load the common Mall Customers dataset.
df = pd.read_csv("Mall_Customers.csv")
features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].dropna()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow curve for K = 1 to 10.
inertia_values = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia_values.append(km.inertia_)
plt.plot(range(1, 11), inertia_values, marker="o")
plt.xlabel("K")
plt.ylabel("Inertia")
plt.title("Elbow Curve")
plt.show()

k = 5  # Replace with the K suggested by your elbow plot.
model = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = model.fit_predict(X_scaled)

# Plot each customer and the centroid locations in original units.
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=labels)
centres = scaler.inverse_transform(model.cluster_centers_)
plt.scatter(centres[:, 0], centres[:, 1], marker="X", s=200, label="Centroids")
plt.xlabel(features[0])
plt.ylabel(features[1])
plt.legend()
```

```
plt.title("Mall Customer Clusters")
plt.show()

print("Number of customers in each cluster:")
print(pd.Series(labels).value_counts().sort_index())
```

### Important lines explained

- `CustomerID` and demographic fields are not used because the problem asks to cluster by income and spending score.
- `fit_predict()` returns one cluster label per row.
- `inverse_transform()` converts the standardized centroid values into familiar income and score units.
- `value_counts()` counts customers assigned to each cluster.
- Cluster numbers such as 0, 1 and 2 are arbitrary labels, not rankings.

### Alternatives and exam-friendly variations

- Try clustering with Age and Spending Score as a separate experiment.
- Use silhouette score to assess cluster quality.
- Use pair plots or summary tables to interpret the characteristics of each customer group.

### Likely viva questions and answers

**Q. Why drop CustomerID?** A. It is an identifier and does not describe customer behaviour.

**Q. What are the two clustering features?** A. Annual income and spending score.

**Q. What does K mean?** A. The number of clusters requested.

**Q. What does a centroid represent?** A. The average standardized feature position of a cluster.

## Q28. Evaluate K-Means for Different Cluster Counts

**Problem focus:** Evaluate K-Means for Different Cluster Counts.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

K-Means performance depends on the selected number of clusters K. Inertia always tends to decrease as K increases, so it should be interpreted using the elbow method rather than minimized blindly. The silhouette score measures how close a point is to its own cluster compared with other clusters; higher values generally indicate better separation. Features should be cleaned and standardized. PCA can be used to visualize the final clusters when the dataset has three or more features.

### Beginner-level Python program (with comments)

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# Iris provides four numerical features, so no CSV is required.
data = load_iris()
X = data.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

inertias = []
silhouette_values = []
ks = [2, 3, 4, 5]
```

```
for k in ks:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    inertias.append(model.inertia_)
    silhouette_values.append(silhouette_score(X_scaled, labels))
    print("K =", k, "Inertia =", model.inertia_,
          "Silhouette =", silhouette_values[-1])

plt.plot(ks, inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Evaluation")
plt.show()

plt.plot(ks, silhouette_values, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Silhouette score")
plt.title("Silhouette Evaluation")
plt.show()

# Choose a final K after comparing both evaluation results.
chosen_k = 3
final_model = KMeans(n_clusters=chosen_k, random_state=42, n_init=10)
final_labels = final_model.fit_predict(X_scaled)

# Project the standardized four-dimensional features into two dimensions.
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=final_labels)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Final K-Means Clusters")
plt.show()
```

### Important lines explained

- The Iris dataset contains four numerical inputs and is used here as an example dataset.
- `ks` lists the cluster counts to evaluate.
- `inertia_` measures within-cluster squared distance.
- `silhouette_score()` evaluates separation for K values of at least 2.
- The two plots give complementary evidence; choose K by interpreting them, not by taking the lowest inertia.
- PCA is used only to display the final clusters in two dimensions.

### Alternatives and exam-friendly variations

- Use another numerical dataset, such as the Wine dataset, by replacing `load_iris()` with `load_wine()`.
- Try Calinski-Harabasz or Davies-Bouldin scores as additional cluster metrics.
- Use a different K only after considering the elbow, silhouette and practical meaning of the groups.

### Likely viva questions and answers

**Q. Why not select the largest K?** A. Because inertia decreases with K, and too many clusters can simply split natural groups unnecessarily.

**Q. What is silhouette score?** A. A measure of how well a point fits its own cluster compared with the nearest other cluster.

**Q. What does PCA do here?** A. It reduces the feature space to two coordinates for visualization.

**Q. Is Iris K-Means supervised?** A. No. The K-Means algorithm is not given the Iris species labels.

# Experiment 7: Apriori Association Rule Mining

## Q29. Market Basket Analysis Using Apriori

**Problem focus:** Market Basket Analysis Using Apriori.

**Dataset / setup:** No external dataset required for this sample; it contains 10 illustrative baskets. Requires `mlxtend` and `pandas`.

### Theory to write in the practical record

Market Basket Analysis discovers items that frequently occur together in transactions. Apriori first identifies frequent itemsets using a minimum support threshold, then derives association rules. Support is the proportion of transactions containing both the antecedent and consequent. Confidence is the proportion of transactions containing the antecedent that also contain the consequent. Lift compares the rule's confidence with the consequent's overall frequency; lift greater than 1 indicates a positive association in the data. Association does not prove causation.

### Beginner-level Python program (with comments)

```
# Install the library once if needed: pip install mlxtend pandas
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Each inner list represents one customer\'s basket (at least 10 transactions).
transactions = [
    ["Milk", "Bread", "Butter"],
    ["Bread", "Diapers", "Beer"],
    ["Milk", "Bread", "Diapers", "Beer"],
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter"],
    ["Milk", "Diapers", "Beer"],
    ["Bread", "Diapers"],
    ["Milk", "Bread", "Butter", "Diapers"],
    ["Milk", "Bread"],
    ["Diapers", "Beer"]
]

# Convert baskets into a True/False table: one column for each item.
encoder = TransactionEncoder()
encoded = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded, columns=encoder.columns_)
print("One-hot transaction table:\n", df)

# Find item combinations appearing in at least 20% of baskets.
frequent_items = apriori(df, min_support=0.2, use_colnames=True)
print("Frequent itemsets:\n", frequent_items)

# Generate rules whose confidence is at least 50%.
rules = association_rules(frequent_items, metric="confidence", min_threshold=0.5)
print("Association rules:")
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])
```

### Important lines explained

- `transactions` is a list of baskets; each inner list contains items bought together.
- `TransactionEncoder` creates one Boolean column per unique item.
- `fit()` learns the item vocabulary and `transform()` makes the one-hot matrix.
- `apriori()` finds itemsets whose support is at least 0.2.

- `association_rules()` creates rules that meet the chosen confidence threshold.
- The output columns include support, confidence and lift.

### Alternatives and exam-friendly variations

- Lower `min_support` to find more rare itemsets; this can increase computation.
- Use `metric="lift"` and a threshold when lift is the main criterion for selecting rules.
- Use real store transactions instead of the sample list when a dataset is provided.

### Likely viva questions and answers

**Q. What is support?** A. The proportion of all transactions containing an itemset.

**Q. What is confidence?** A. The conditional proportion of transactions with the antecedent that also contain the consequent.

**Q. What is lift?** A. Confidence divided by the overall support of the consequent.

**Q. Does a high lift prove causality?** A. No. It indicates association in the observed transaction data, not causation.

# Experiment 8: Artificial Neural Networks

## Q30. Bank Customer Churn Prediction Using ANN

**Problem focus:** Bank Customer Churn Prediction Using ANN.

**Dataset / setup:** Bank Customer Churn Modeling CSV. Expected target `Exited`; common file name `Churn_Modelling.csv`.

### Theory to write in the practical record

An Artificial Neural Network (ANN) is a model made of interconnected computational units called neurons. A feed-forward ANN has an input layer, one or more hidden layers and an output layer. Hidden layers commonly use ReLU activation, while a binary output layer uses sigmoid to produce a probability. Binary cross-entropy is a standard loss function for binary classification. Numerical inputs should be scaled and categorical inputs encoded before training. The test set is used only for final evaluation.

### Beginner-level Python program (with comments)

```
# Install TensorFlow if needed: pip install tensorflow
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Expected Kaggle Bank Churn CSV with target column Exited (0/1).
df = pd.read_csv("Churn_Modelling.csv")
# Remove identifier/name columns if present.
remove_columns = [c for c in ["RowNumber", "CustomerId", "Surname"] if c in df.columns]
df = df.drop(columns=remove_columns)

y = df["Exited"]
X = df.drop(columns=["Exited"])
# Convert Geography and Gender (and any other text columns) into numeric columns.
X = pd.get_dummies(X, drop_first=True, dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build a small feed-forward neural network.
model = Sequential([
    Dense(16, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid")
])

# Configure training for binary classification.
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train_scaled, y_train, epochs=20, batch_size=32, validation_split=0.1)

# Predict probabilities, then convert them to 0/1 labels.
probabilities = model.predict(X_test_scaled)
predictions = (probabilities >= 0.5).astype(int).ravel()
print("Accuracy:", accuracy_score(y_test, predictions))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- Identifier columns are removed because they are not useful general predictors.
- `get_dummies()` converts categorical columns to numeric features.
- `StandardScaler` is fitted only on training features.
- `Sequential` defines a simple stack of layers.
- The first Dense layer has 16 neurons, the second has 8, and the output has one neuron.
- `relu` is used in hidden layers; `sigmoid` outputs a probability between 0 and 1.
- `binary_crossentropy` is the loss for binary labels; Adam is the optimizer.
- `predict()` returns probabilities; threshold 0.5 converts them to class labels.

### Alternatives and exam-friendly variations

- Use `Dropout(0.2)` between hidden layers to reduce overfitting.
- Use early stopping based on validation loss.
- Use class weights or a precision-recall threshold when churn classes are imbalanced.
- In newer Keras versions, an explicit `Input(shape=(n_features,))` layer can replace `input_shape` in the first Dense layer.

### Likely viva questions and answers

**Q. What is an ANN?** A. A network of connected neurons that learns patterns by adjusting weights.

**Q. What is an epoch?** A. One complete pass through the training dataset.

**Q. Why ReLU?** A. It is a simple non-linear activation commonly used in hidden layers.

**Q. Why sigmoid output?** A. It converts the output into a probability-like value between 0 and 1.

## Q31. Student Pass/Fail Prediction Using ANN

**Problem focus:** Student Pass/Fail Prediction Using ANN.

**Dataset / setup:** Student performance CSV. Expected inputs: `attendance`, `study_hours`, `internal_marks`, `assignment_score`; target is derived from `final_score >= 40`.

### Theory to write in the practical record

Pass/fail prediction is a binary classification task. Student attributes such as attendance, study hours, internal marks and assignment scores form the input features. A feed-forward ANN learns a non-linear relationship between these features and the pass/fail label. The numerical input features are standardized, and a sigmoid output neuron predicts the probability of passing. Accuracy, precision, recall and a confusion matrix evaluate the predictions.

### Beginner-level Python program (with comments)

```
# Install TensorFlow if needed: pip install tensorflow
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Expected columns: attendance, study_hours, internal_marks, assignment_score, final_score.
df = pd.read_csv("student_performance.csv")
features = ["attendance", "study_hours", "internal_marks", "assignment_score"]
X = df[features].fillna(df[features].median())
# Create the binary target: final score >= 40 means pass in this example.
y = (df["final_score"] >= 40).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = Sequential([
    Dense(16, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train_scaled, y_train, epochs=30, batch_size=16, validation_split=0.1)

probabilities = model.predict(X_test_scaled)
predictions = (probabilities >= 0.5).astype(int).ravel()
print("Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions, zero_division=0))
print("Recall:", recall_score(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- `features` contains the student attributes used for prediction; the final score is not included as an input.
- `y` converts the final score into 1 (pass) or 0 (fail) using the stated threshold of 40.
- `StandardScaler` normalizes the input features using training-set statistics.
- The network contains two hidden layers and one sigmoid output neuron.
- `epochs=30` controls how many times the network sees the training data.
- A probability of 0.5 or more is labelled pass; below 0.5 is labelled fail.

### Alternatives and exam-friendly variations

- Use an existing `Pass_Fail` column instead of creating one from `final_score` if the dataset already supplies it.
- Try a different pass threshold if the academic policy defines one.
- Use early stopping or dropout if training accuracy rises while validation performance worsens.

### Likely viva questions and answers

**Q. What is binary classification?** A. Predicting one of two possible classes.

**Q. Why not include final_score in X?** A. Because it is used to create the target; including it would leak the answer into the input.

**Q. What does the sigmoid layer produce?** A. A value between 0 and 1 used as the estimated probability of the positive class.

**Q. What is data normalization?** A. Rescaling numerical features so their values are on comparable scales.

# Experiment 9: Integrated Classification and ML Pipeline

## Q32. Compare Decision Tree, KNN and Naive Bayes

**Problem focus:** Compare Decision Tree, KNN and Naive Bayes.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

This experiment compares three supervised classification algorithms on the same labelled dataset. Decision Tree learns feature-based rules, KNN classifies using nearby training samples, and Gaussian Naive Bayes applies Bayes theorem with a conditional-independence assumption and Gaussian feature likelihoods. The data split should be stratified. KNN requires feature scaling, while the tree does not depend on scale in the same way. Accuracy, precision, recall, F1-score and confusion matrices provide a common comparison.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# A labelled, two-class dataset is loaded from scikit-learn.
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Pipeline scales features for KNN; it is also harmless for GaussianNB.
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "Gaussian Naive Bayes": make_pipeline(StandardScaler(), GaussianNB())
}

results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions, zero_division=0)
    recall = recall_score(y_test, predictions, zero_division=0)
    f1 = f1_score(y_test, predictions, zero_division=0)
    results.append([name, accuracy, precision, recall, f1])
    print("\n", name)
    print("Confusion matrix:\n", confusion_matrix(y_test, predictions))

# Display a simple comparison table.
print("\nModel comparison:")
print("Model | Accuracy | Precision | Recall | F1")
for row in results:
    print(row[0], *[round(value, 3) for value in row[1:]], sep=" | ")
```

### Important lines explained

- The same stratified split is passed to all three algorithms.
- `DecisionTreeClassifier` does not require feature scaling for its split decisions.

- `make_pipeline()` joins StandardScaler and KNN/GaussianNB so scaling is fitted on training data only.
- `fit()` trains each model; `predict()` produces labels for the same test records.
- `results` stores the four evaluation metrics for a comparison table.
- The best model should be selected based on the problem's error costs, not accuracy alone.

### Alternatives and exam-friendly variations

- Use `MultinomialNB` for non-negative count features such as text word counts; for the numerical breast cancer dataset, GaussianNB is suitable.
- Add cross-validation to estimate performance more robustly.
- Use ROC-AUC or macro F1 if the class distribution or error costs require more detail.

### Likely viva questions and answers

**Q. Why compare on the same split?** A. So each model is evaluated on identical test examples.

**Q. Which algorithm needs scaling most?** A. KNN, because it relies on feature distances.

**Q. What does precision measure?** A. Of the records predicted positive, the proportion that are truly positive.

**Q. What does recall measure?** A. Of the truly positive records, the proportion correctly identified.

## Q33. End-to-End Machine Learning Pipeline

**Problem focus:** End-to-End Machine Learning Pipeline.

**Dataset / setup:** No external dataset required.

### Theory to write in the practical record

An end-to-end machine-learning pipeline includes defining a prediction objective, inspecting data, preparing features, splitting data, training a model, tuning hyperparameters and evaluating the final result. A pipeline ensures that preprocessing steps such as imputation and scaling are fitted only on training folds, which helps prevent data leakage. Cross-validation evaluates hyperparameters over multiple training/validation splits. The final test set is kept separate until model selection is complete.

### Beginner-level Python program (with comments)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Step 1: Choose a labelled dataset and define the target.
data = load_breast_cancer()
X, y = data.data, data.target
print("Dataset shape:", X.shape)
print("Class labels:", data.target_names)

# Step 2: Create a stratified hold-out test set.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Step 3: Define preprocessing and model as one pipeline.
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=2000))
])
```

```
# Step 4: Tune one hyperparameter using cross-validation on training data only.
parameter_grid = {"classifier__C": [0.01, 0.1, 1, 10]}
search = GridSearchCV(pipeline, parameter_grid, cv=5, scoring="accuracy")
search.fit(X_train, y_train)

# Step 5: Evaluate the selected pipeline once on the hold-out test set.
best_model = search.best_estimator_
predictions = best_model.predict(X_test)
print("Best parameters:", search.best_params_)
print("Best cross-validation accuracy:", search.best_score_)
print("Test accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions))
print("Recall:", recall_score(y_test, predictions))
print("F1-score:", f1_score(y_test, predictions))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
```

### Important lines explained

- The built-in Breast Cancer dataset is labelled and numerical, so an external CSV is not required.
- `train_test_split(..., stratify=y)` reserves an independent test set and preserves class balance.
- `Pipeline` applies imputation, scaling and classification in the correct order.
- `SimpleImputer` is included even though this built-in dataset is generally complete, so the pipeline demonstrates missing-value handling.
- `classifier__C` addresses the Logistic Regression C parameter inside the named pipeline step.
- `GridSearchCV` uses cross-validation on training data only.
- The final test evaluation is performed after choosing the best cross-validation model.

### Alternatives and exam-friendly variations

- For a categorical dataset, add a `ColumnTransformer` containing `SimpleImputer` and `OneHotEncoder` for categorical columns.
- Replace Logistic Regression with `RandomForestClassifier` and tune `n_estimators` or `max_depth`.
- Use `RandomizedSearchCV` for a large hyperparameter space.
- The example dataset contains numerical features only, so categorical encoding is not needed; mention this in the practical record rather than inventing categorical columns.

### Likely viva questions and answers

**Q. What is an ML pipeline?** A. A sequence of preprocessing and modeling steps packaged into one workflow.

**Q. What is data leakage?** A. When information from outside the training process, especially the test set, influences model fitting.

**Q. What is cross-validation?** A. Training and validating on several folds to estimate how well parameter choices generalize.

**Q. Why keep a final test set?** A. It provides an independent final estimate after model selection.

# Last-minute oral exam revision

**A* vs Greedy Best-First:** A* selects using g+h; Greedy selects using h only. A* can be optimal with a suitable heuristic; Greedy is not guaranteed optimal.

**CSP:** Name the variables, domains and constraints. Backtracking tries an assignment, checks constraints and undoes failed choices.

**Minimax:** Terminal score -> explore legal moves -> maximize for one player -> minimize for the opponent. Alpha-beta pruning skips irrelevant branches.

**Regression metrics:** R2: explained variation. MAE: average absolute error. RMSE: square root of average squared error.

**Classification metrics:** Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 = harmonic mean of precision and recall.

**Decision Tree:** A tree of if/else splits. Gini and entropy measure impurity. max_depth and min_samples_leaf control complexity.

**KNN:** Distance-based; K is the neighbour count; scaling matters; predictions use voting for classification.

**Naive Bayes:** Bayes theorem plus conditional independence assumption. GaussianNB suits continuous features; MultinomialNB suits word counts.

**Random Forest:** Many randomized decision trees combined by voting; usually more stable than one deep tree.

**K-Means:** Unsupervised clustering. K is specified; centroids update; inertia is within-cluster squared error; elbow/silhouette help choose K.

**Apriori:** Find frequent itemsets, then rules. Support = joint frequency; confidence = conditional frequency; lift = confidence / consequent support.

**ANN:** Input layer -> hidden layer(s) -> output. ReLU is common in hidden layers; sigmoid for binary output; binary cross-entropy for binary labels.

**PCA:** Standardize first; PCA forms new orthogonal components; fit it on training data only; dimensionality reduction may help or hurt model performance.

**Data leakage:** Never fit a scaler, imputer, encoder or PCA on the full dataset before splitting. Fit on training data, transform test data.

## Important dataset reminder

The supplied problem sheet names external Kaggle datasets for some questions but does not include those CSV files. Therefore, the CSV-based programs in this guide show explicit expected filenames and column headers. Before the exam, open your lab dataset, check its actual headers with print(df.columns), and update the feature/target names to match. The built-in scikit-learn datasets are used where they make the practical self-contained.